```
        </div>
        <button class="btn btn-primary">Submit</button>
    </form>
    @if(Request.HasFormContentType &&
    ➥ !string.IsNullOrWhiteSpace(Model.CityName))
    {
        <p>You submitted @Model.CityName</p>
    }
</div>
```

As before, if you run the page it should produce the same output when the form is submitted.

The handler parameter has been replaced with a public property with the same name. The `CityName` property differs from the removed `Message` property in that it is decorated with the `BindProperty` attribute, which specifies that the property is a binding target. Without this attribute, the property will not take part in model binding.

By default, the `BindProperty` attribute only opts properties in to model binding for `POST` requests. If you want to bind values to public properties during `GET` requests (e.g., from route data or the query string), an extra step is required to opt in. The `BindProperty` attribute has a property named `SupportsGet`, which you must set to `true`:

```
[BindProperty(SupportsGet=true)]
public int Id  { get; set; }
```

In addition to the `SupportsGet` property, the `BindProperty` attribute has a `Name` property, which enables you to bind a public `PageModel` property to an arbitrarily named form field. For example, you might need to bind from a form control that has a `name` attribute set to `"e-mail"`. That's not a valid C# identifier, so you use the `Name` property to map the incoming form value to a property C# will tolerate: